28 July 2026

PROJECT BLACK CHALLENGE 5

Bian Yuxuan Jonathan

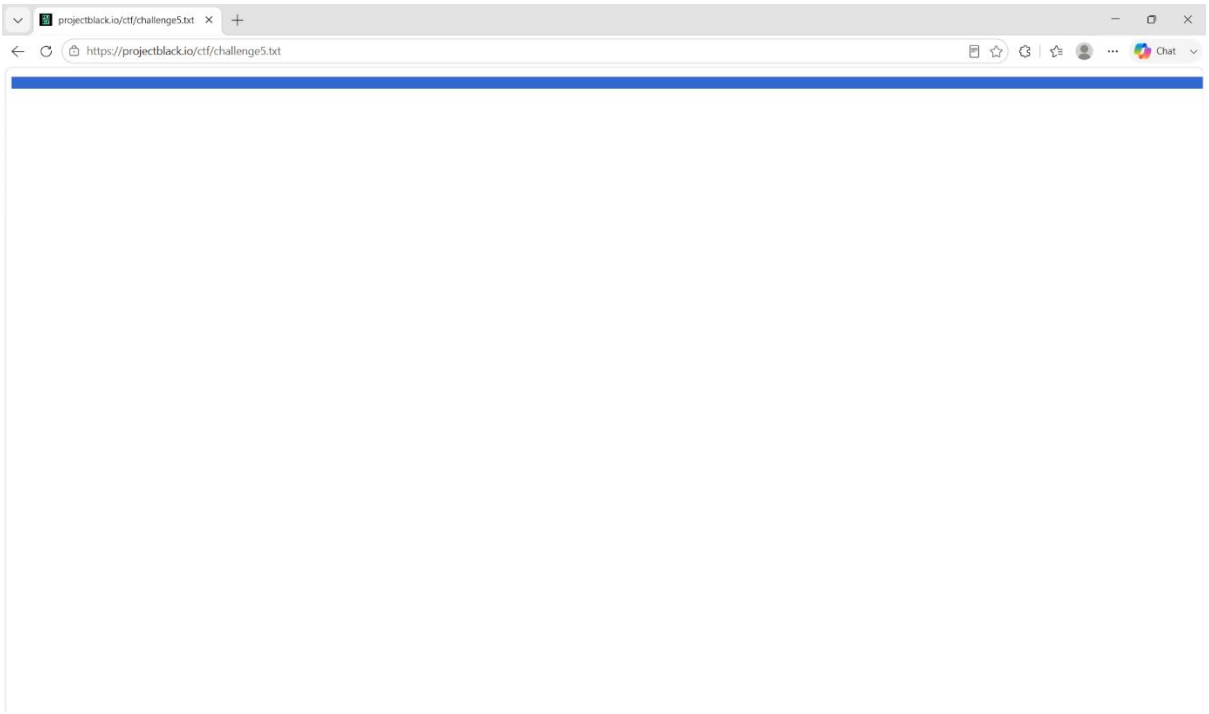
University of Technology Sydney

Introduction

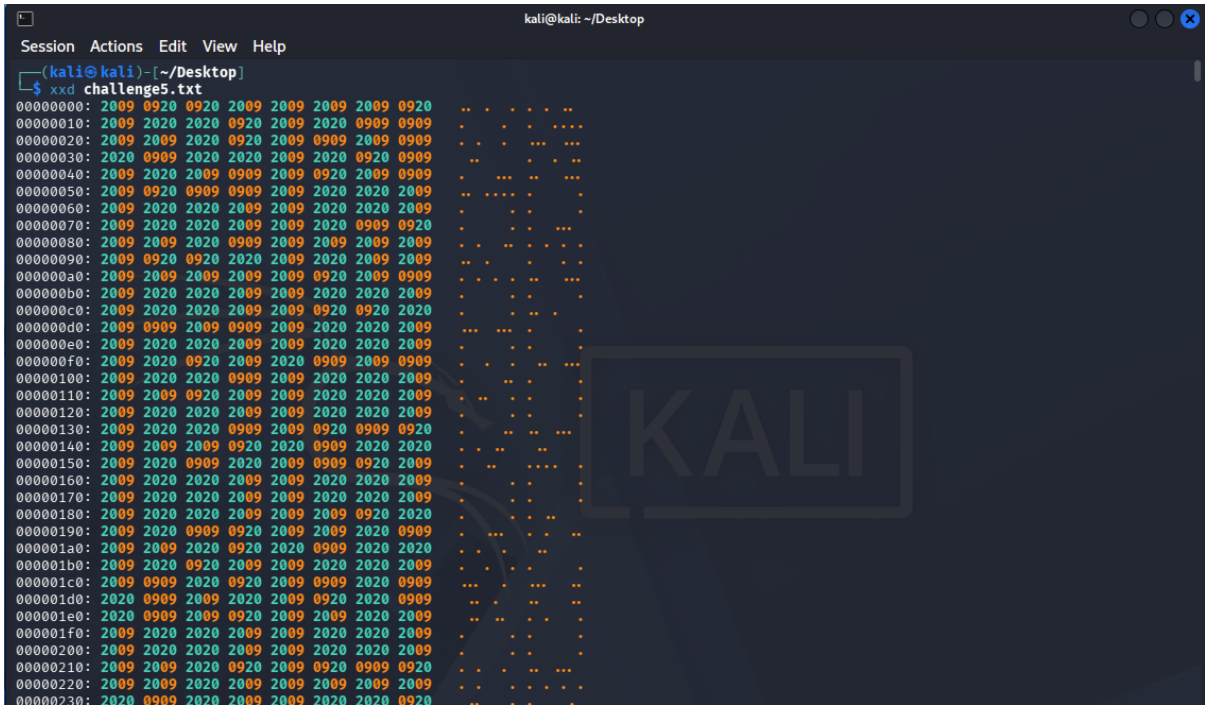
This report documents the findings from a Capture The Flag (CTF) challenge completed as part of a recruitment process by Project Black. The exercise was designed to assess candidates' offensive security skills in a controlled environment.

The challenge consisted of two components: a digital forensics task involving steganographic encoding, and a vulnerable web application assessment. Both components required identifying, analysing, and exploiting security weaknesses to achieve the intended objectives.

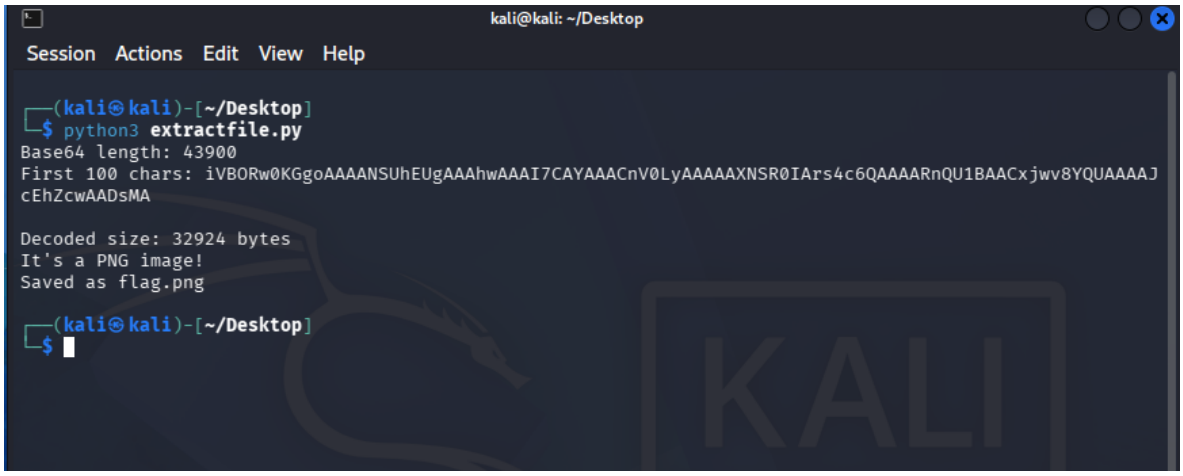
Challenge 1: Digital Forensics and Steganography



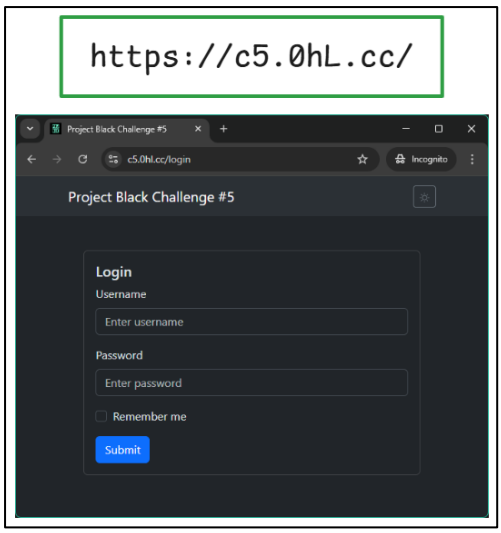
The website features challenge5.txt, which contains invisible characters.



Upon inspecting the file using a hex editor, I observed that although the file appeared empty due to the presence of invisible characters, its contents consisted entirely of ASCII values 0x09 (Tab) and 0x20 (Space). These characters correspond to whitespace-based encoding, which led me to identify a potential use of the Whitespace esoteric programming language, a technique commonly encountered in CTF challenges, particularly within the miscellaneous category.



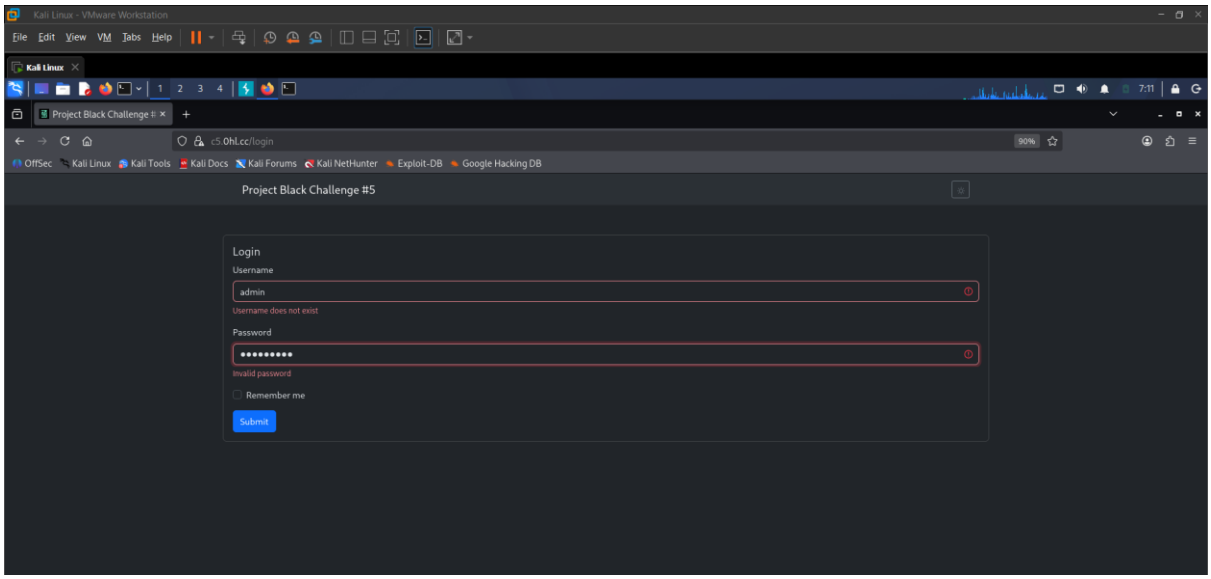
I developed a Python script to extract hidden data from challenge5.txt, which contained information encoded using whitespace steganography. The script interprets tab characters as binary 1 values and space characters as binary 0 values to reconstruct the original byte sequence. The reconstructed bytes are then processed as a Base64-encoded string and decoded to recover the underlying binary data. Finally, the script analyses the file signature (magic bytes) to identify the recovered data as a PNG image and saves it with the .png file extension.



From the recovered image, I identified a URL that directed to the next challenge stage. During the analysis process, multiple decoding approaches were investigated. Initially, I attempted to convert the file into a .ws format and decode it using a Whitespace language decoder; however, this approach did not produce meaningful results. Further examination of the file contents within a hex editor revealed a pattern of dots and spaces, which suggested a possible Morse code encoding. Decoding this pattern successfully revealed the URL required to proceed to the next challenge.

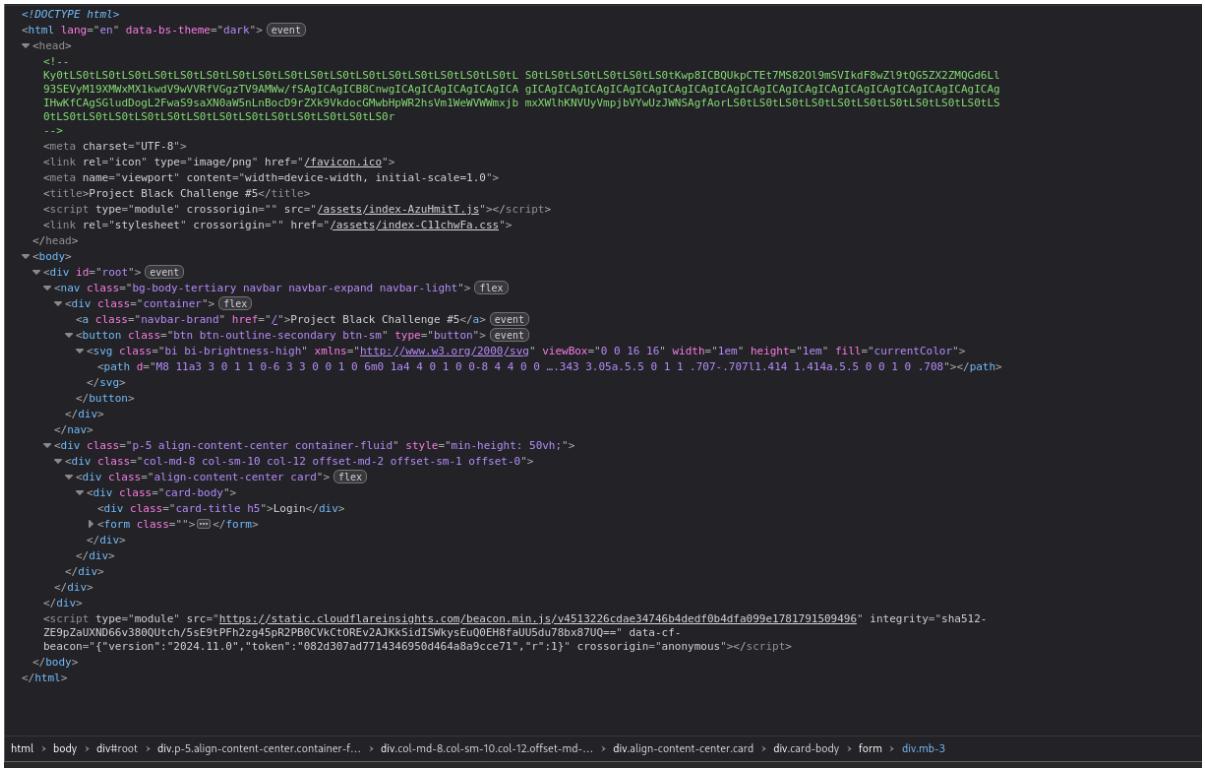
Challenge 2: Web

Part 2 of the challenge introduced a login page. During the initial assessment, I tested the input fields for potential SQL injection vulnerabilities, and I identified two key observations:



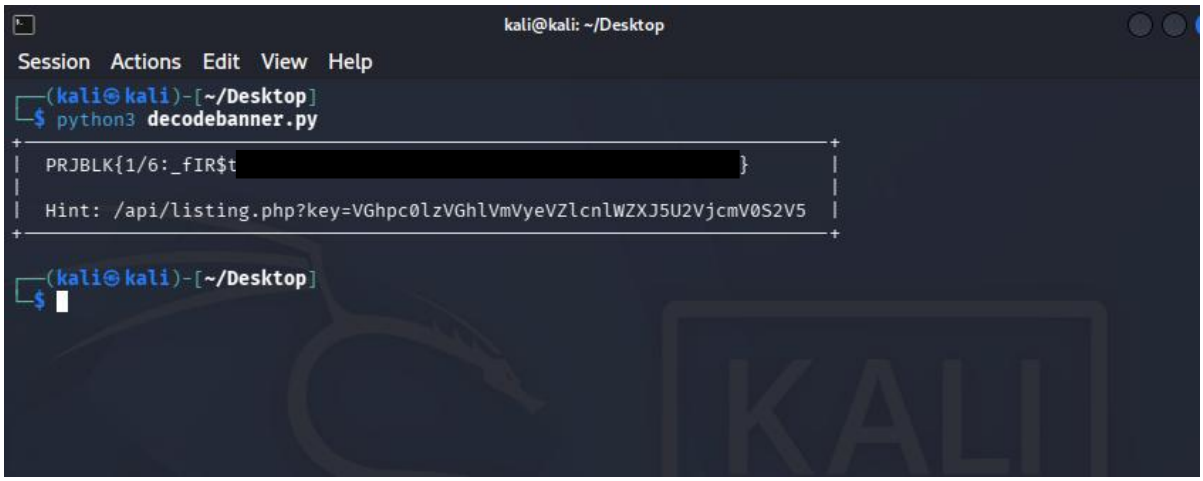
1. The application performs validation checks to determine the validity of supplied username.

2. The application performs password validation checks against the provided credentials.

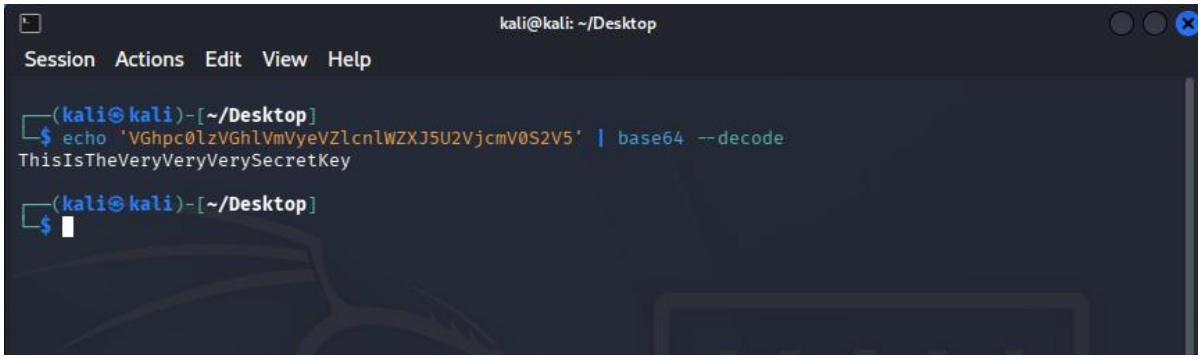


Upon further enumeration, I identified a Base64-encoded string embedded within an HTML comment. I used a Python script to decode the string and reveal the original UTF-8 text.

Flag 1/6

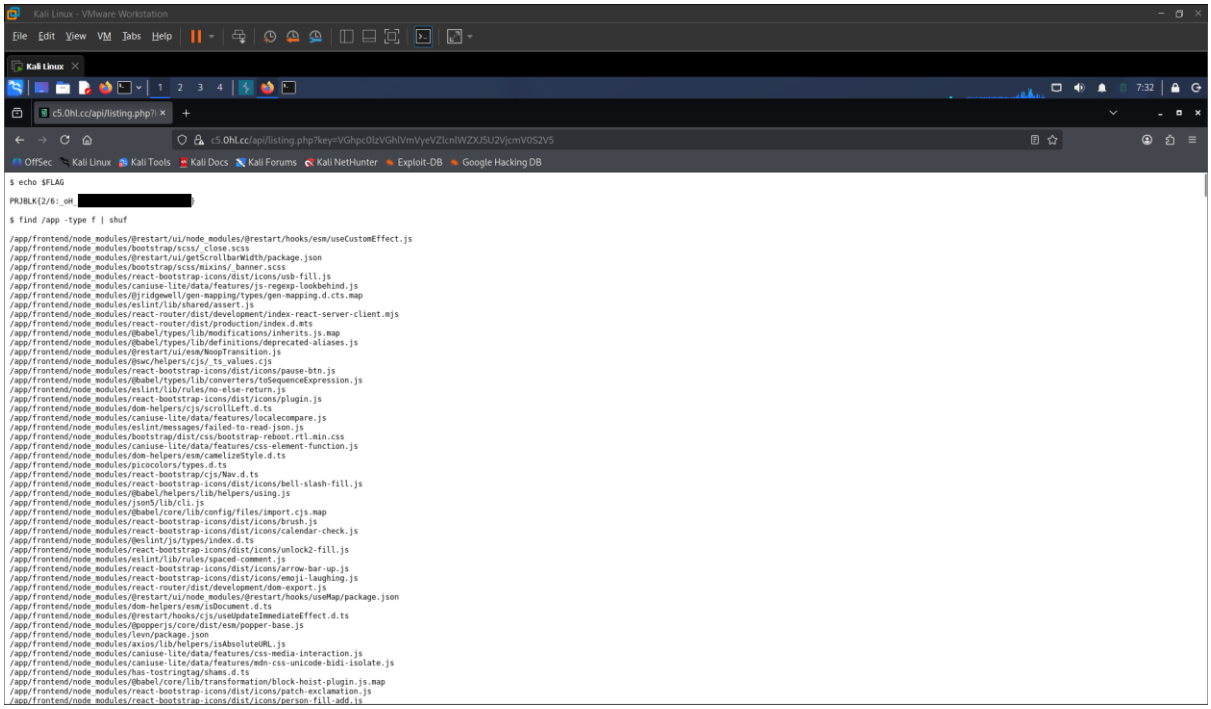


Upon decoding the Base64 string, I recovered the first of six flags. The decoded message also contained a hint which identify an API endpoint used by the web application.



I then decoded the value supplied in the key URL parameter, revealing the string ThisIsTheVeryVeryVerySecretKey. At the time, I did not recognize its significance and continued with further enumeration. This oversight later proved to be important, as the key played a critical role later.

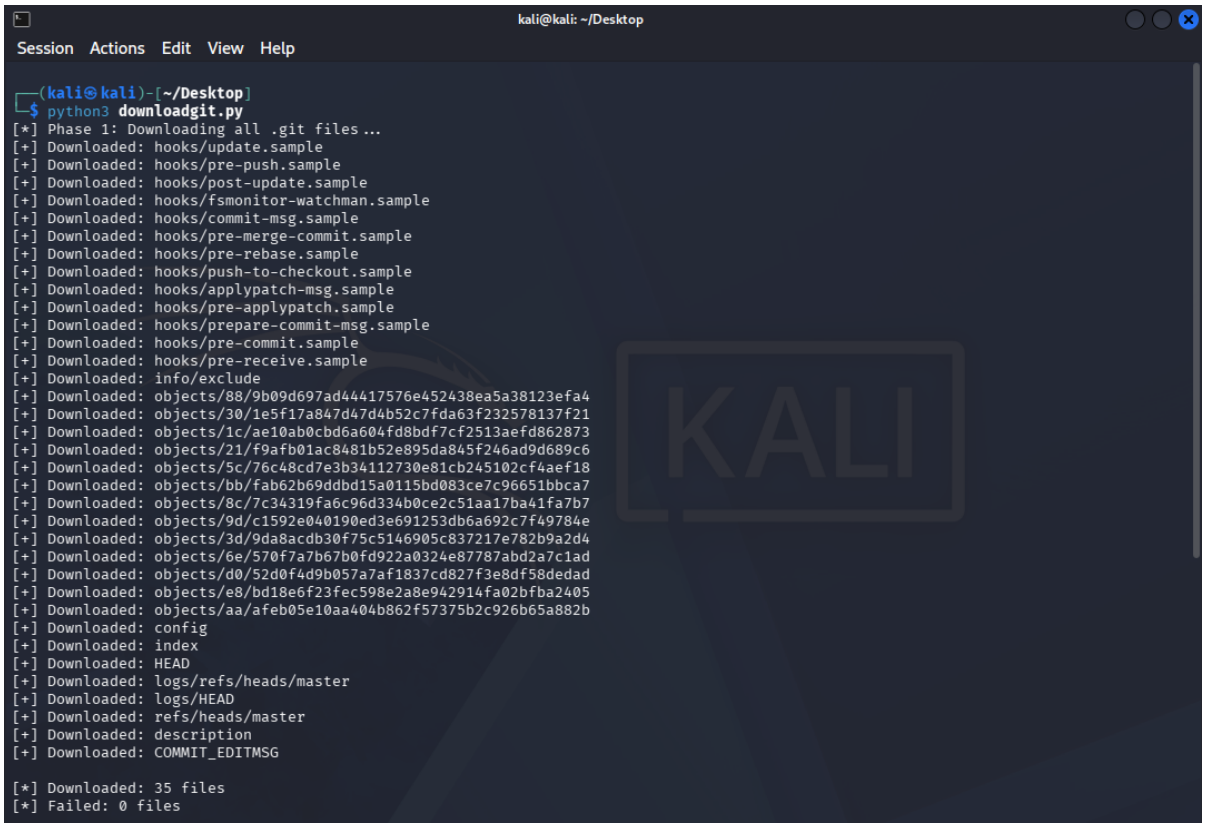
Flag 2/6



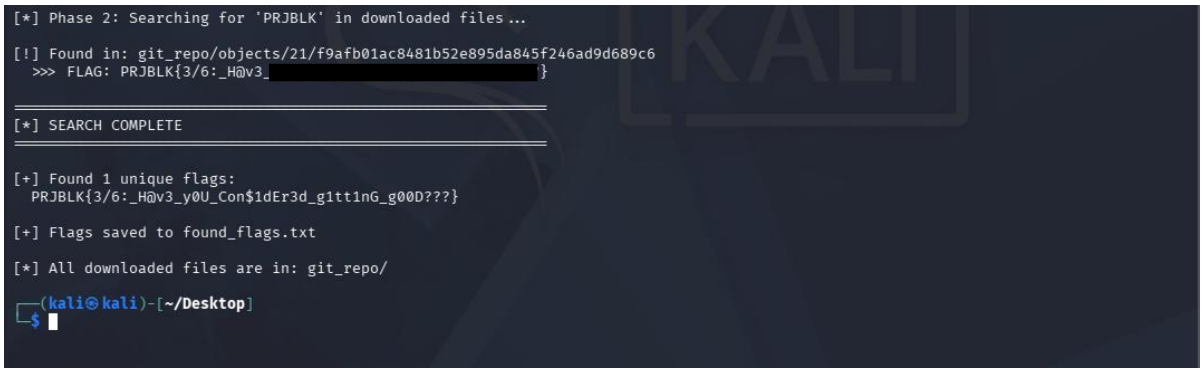
My initial assumption was that the listing.php API endpoint would expose useful information for further enumeration, such as additional API endpoints, usernames, hashed passwords, or other application resources. Guided by the hint obtained earlier, I accessed the endpoint. Contrary to my expectations, the endpoint did not reveal sensitive application data directly. Instead, it presented a directory listing of all files located under the /app parent directory as well as the second of six flags.

Given the large number of files returned, I organized the results into two categories for further analysis: useful API endpoints and the publicly exposed .git directory.

Flag 3/6

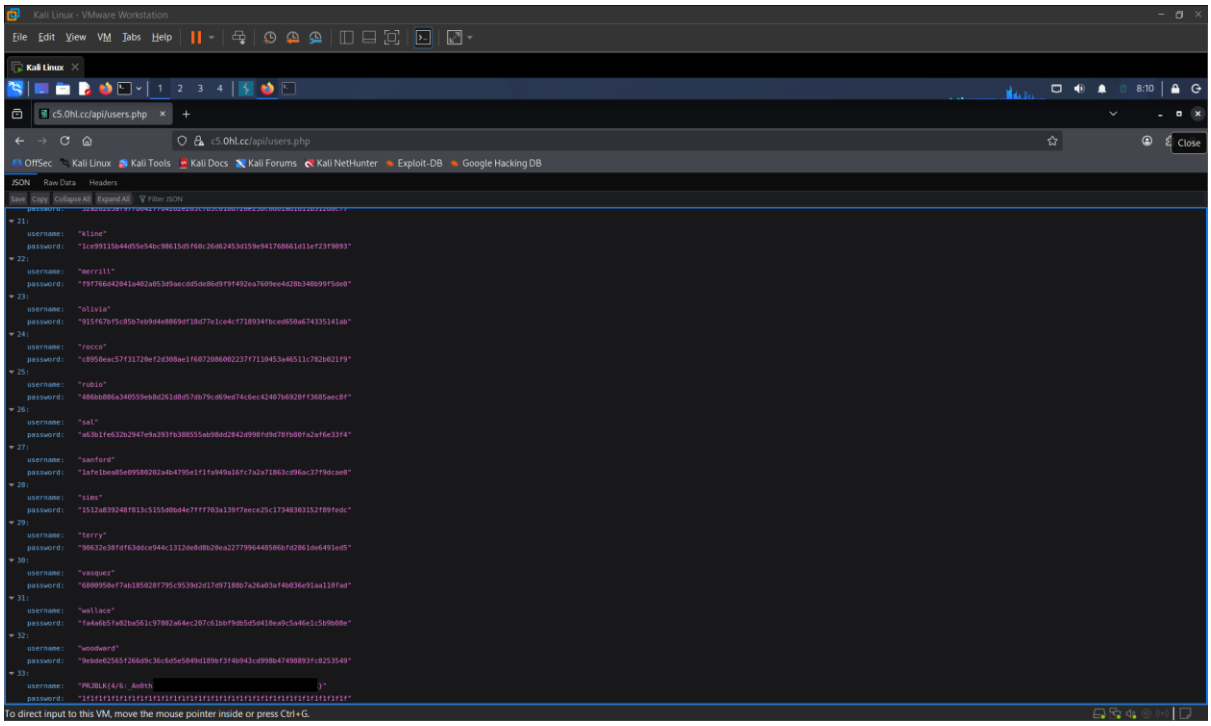


I developed a two-part Python script. The first phase iterates through all exposed .git URL paths and downloads the accessible files associated with the repository structure.



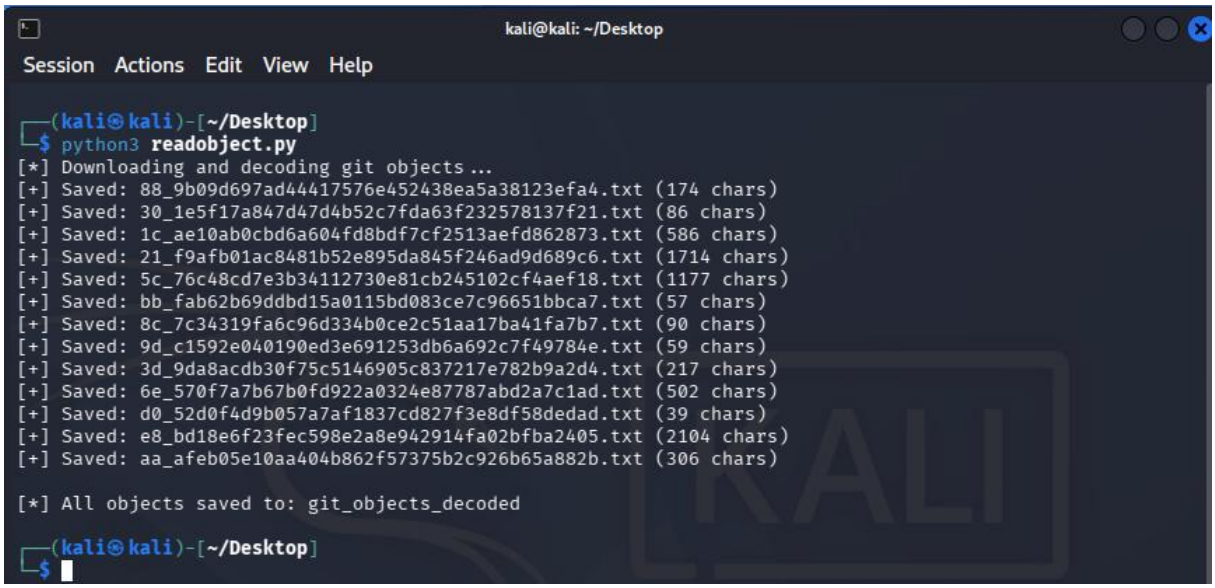
The second phase of the script processes the files downloaded during the initial enumeration phase. It scans the contents of each file and searches for the flag using the known identifier prefix PRJBLK. The third flag is found in file /21/f9afb01ac8481b52e895da845f246ad9d689c6.

Flag 4/6



From the list of available API endpoints, I focused my analysis on users.php, which contained a list of usernames and their corresponding password hashes. Upon reviewing the returned data, I identified the fourth flag within the username field at index 33.

The passwords were stored using the SHA-256 hashing algorithm. I extracted the hashes into a separate file and attempted to recover the plaintext passwords using password-cracking tools, including John the Ripper and Hashcat. Additionally, I submitted the hashes to CrackStation for comparison against known hash databases. However, none of these approaches successfully recovered the original passwords.



I developed a Python script to automate the extraction of exposed Git objects. The script downloads the identified Git objects, decompresses their zlib-compressed contents, and decodes the recovered data into UTF-8 format for further analysis.

```
41 $targetUser = get_user_object($username);
42
43 if ($targetUser === null) {
44     echo json_encode(['status' => 'USER_DOES_NOT_EXIST']);
45     exit();
46 }
47
48 /*
49  * TODO my boss said juggling is bad, don't know what that's
50  *      supposed to be mean bedtime reading:
51  *
52  *   https://www.php.net/manual/en/language.operators.comparison.php
53  *
54  *   PRJBLK{3/6[redacted]}
55  */
56 $password = $userRequest["password"] ?? '';
57 if (hash('sha256', $password) !== $targetUser['password']) {
58     echo json_encode(['status' => 'INVALID_PASSWORD']);
59     exit();
60 }
61
62 echo json_encode([
63     'status' => 'GREAT_SUCCESS',
64     'accessToken' => generate_jwt($username, 'user', 42),
65     'notification' => file_get_contents('/app/config-files/flag5.txt'),
66 ]);
67 exit();
68 ?>
69
```

At line 57 of 21_f9afb01ac8481b52e895da845f246ad9d689c6.txt, the following code was identified:

if (hash('sha256', $password) != $targetUser['password']) {

This line introduces a potential PHP type juggling vulnerability due to the use of a loose comparison operator (!=) instead of a strict comparison operator (!==). In PHP, loose comparisons allow automatic type conversion before evaluating the condition, which can result in unexpected comparison behaviour when handling user-controlled input. This weakness may allow an attacker to bypass authentication checks under specific conditions.

Eddie's password hash:

0ecf0da80c812d7c8fb735be03a80a3fc2b307fbd992fc896bb6a066e953c7d1

PHP's loose comparison operator treats strings starting with "0e" as scientific notation, where "0e" followed by any characters evaluates to 0 (zero).

Strict Comparison	Loose Comparison
"0e12345" === "0e67890" // FALSE (different strings)	"0e12345" == "0e67890" // TRUE! (both are 0)

PHP Scientific Notation

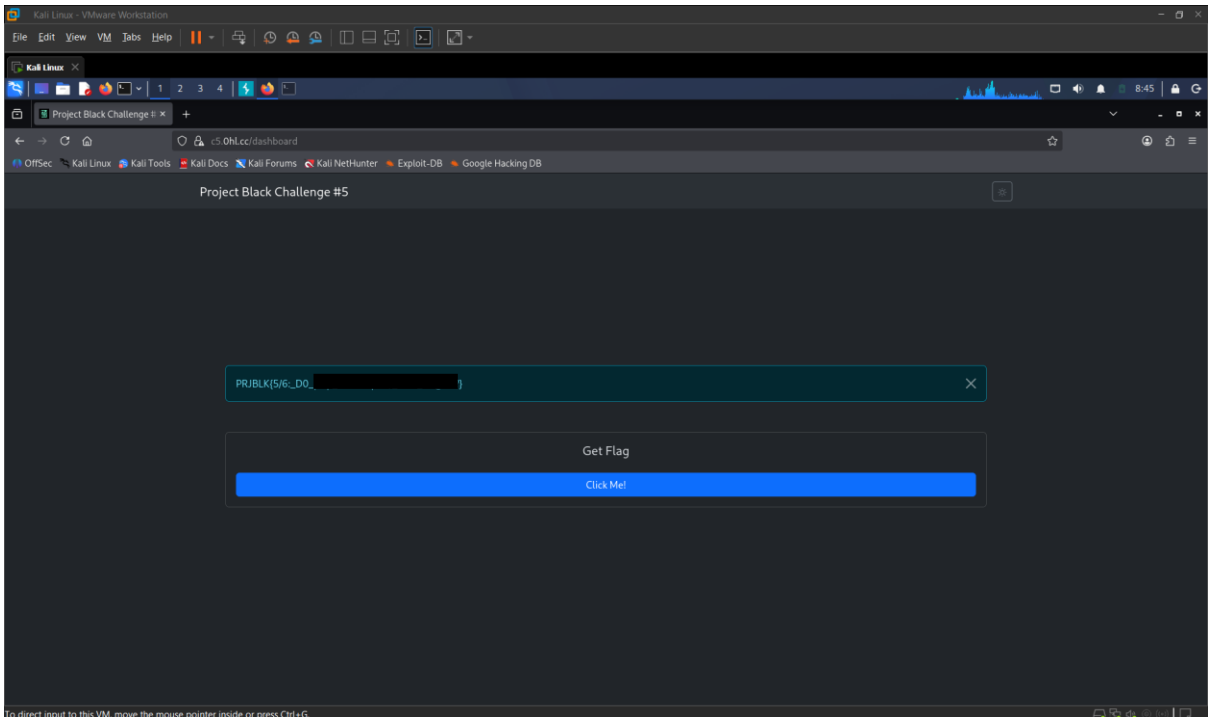
Notation	Means	Value
0e5	0×10^{5}	0
0e123	0×10^{123}	0
0e999	0×10^{999}	0

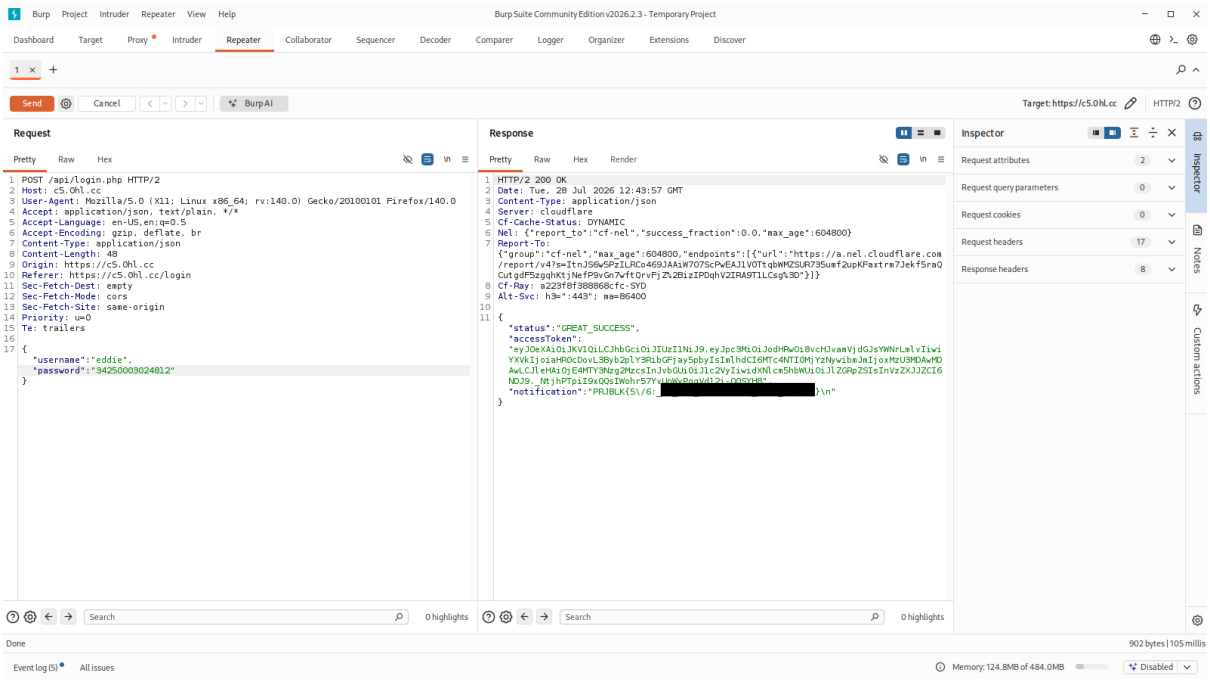
An attacker can exploit this vulnerability by identifying a password input that produces a SHA-256 hash matching the PHP type juggling condition, specifically a hash beginning with 0e followed exclusively by numerical digits. When the application performs a loose comparison using the != operator, PHP may automatically interpret these hash values as numeric values in scientific notation rather than as strings. As a result, two different hash strings can be evaluated as the same numerical value (0). The comparison therefore returns false, causing the password validation condition to be bypassed and allowing the authentication process to proceed without requiring knowledge of the legitimate password.

SHA-256 Payload (Credit: @TihanyiNorbert)

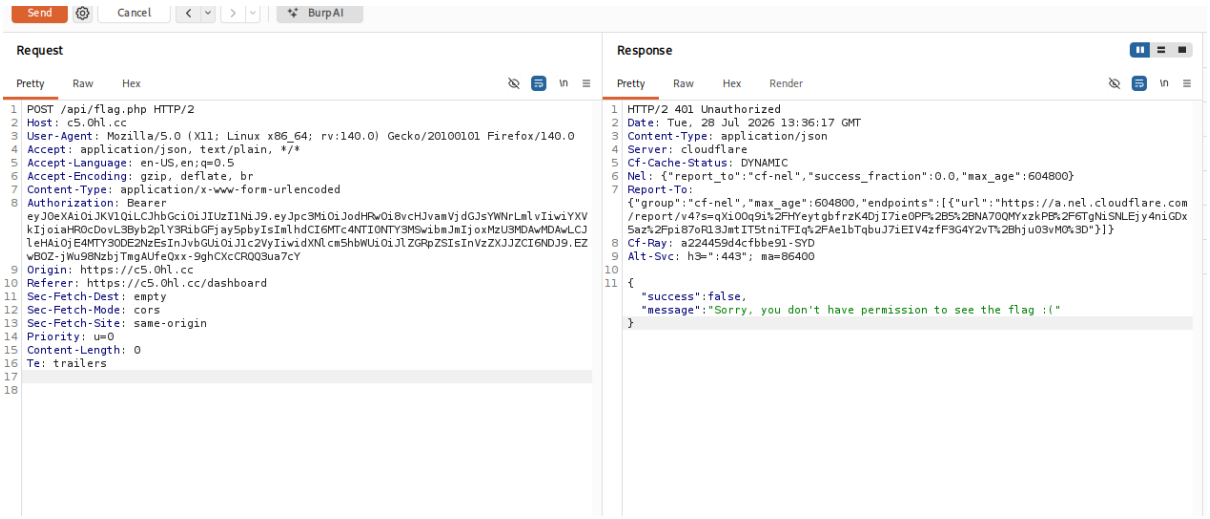
34250003024812	0e462890320380659161396210390858837734138209919207062996950513332

Flag 5/6



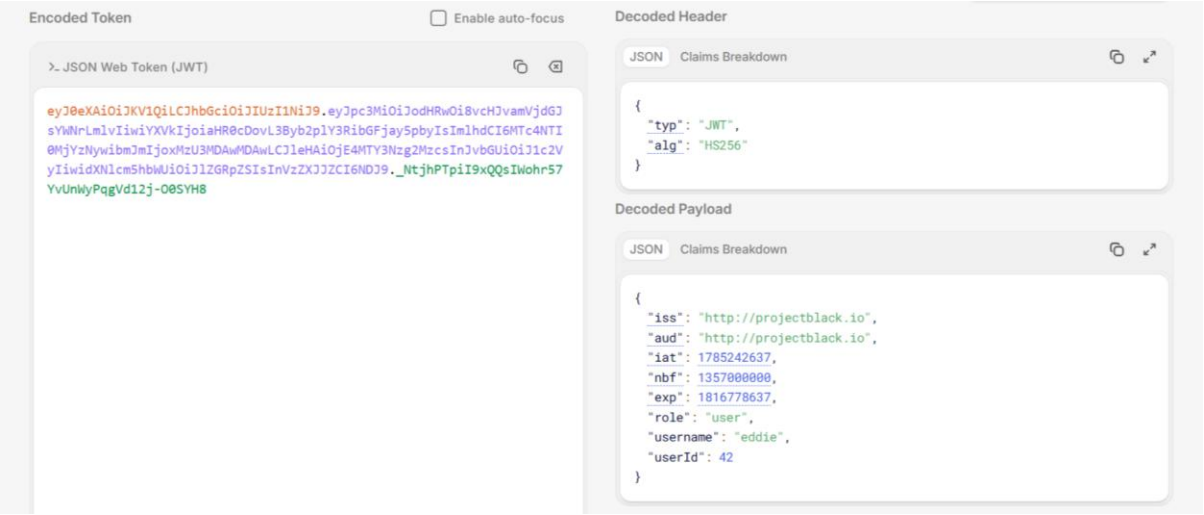


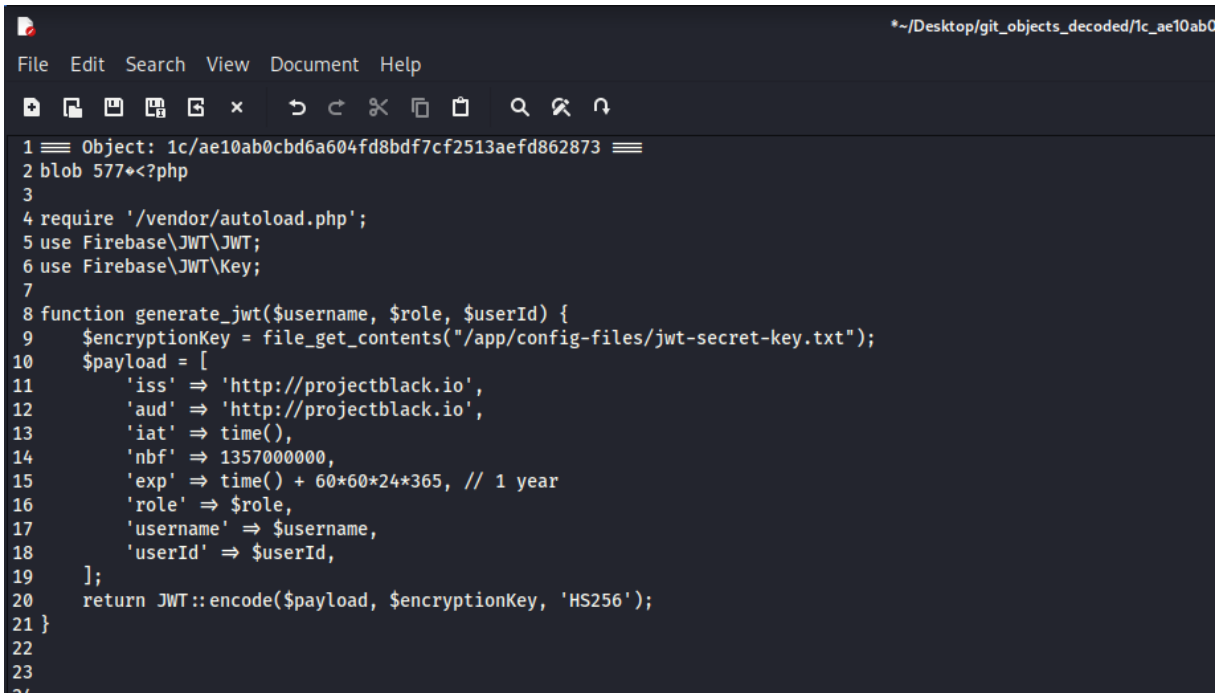
Upon successfully authenticating as Eddie's account, I identified the fifth flag.



The last flag was secured behind the dashboard button and was inaccessible. The button triggered a request to the flag.php API endpoint, which returned an insufficient privileges message. Additionally, I identified an accessToken generated by the jwt.php API endpoint within the HTTP response captured through the BurpSuite proxy, which was subsequently analysed.

JWT Breakdown





Based on the observed application behaviour, the authentication workflow appears to function as follows: after successful authentication, user information is transmitted to the jwt.php API endpoint, which generates a JWT access token associated with the authenticated user. The token is signed using the HS256 algorithm with the secret key stored within jwt-secret-key.txt. Most fields within the JWT header and payload follow standard JWT structures. However, the role, username, and userId fields contain application-specific information that may influence authorization decisions.

```php
$password = $userRequest["password"] ?? '';
if (hash('sha256', $password) ≠ $targetUser['password']) {
    echo json_encode(['status' ⇒ 'INVALID_PASSWORD']);
    exit();
}

echo json_encode([
    'status' ⇒ 'GREAT_SUCCESS',
    'accessToken' ⇒ generate_jwt($username, 'user', 42),
    'notification' ⇒ file_get_contents('/app/config-files/flag5.txt'),
]);
exit();
?>
```

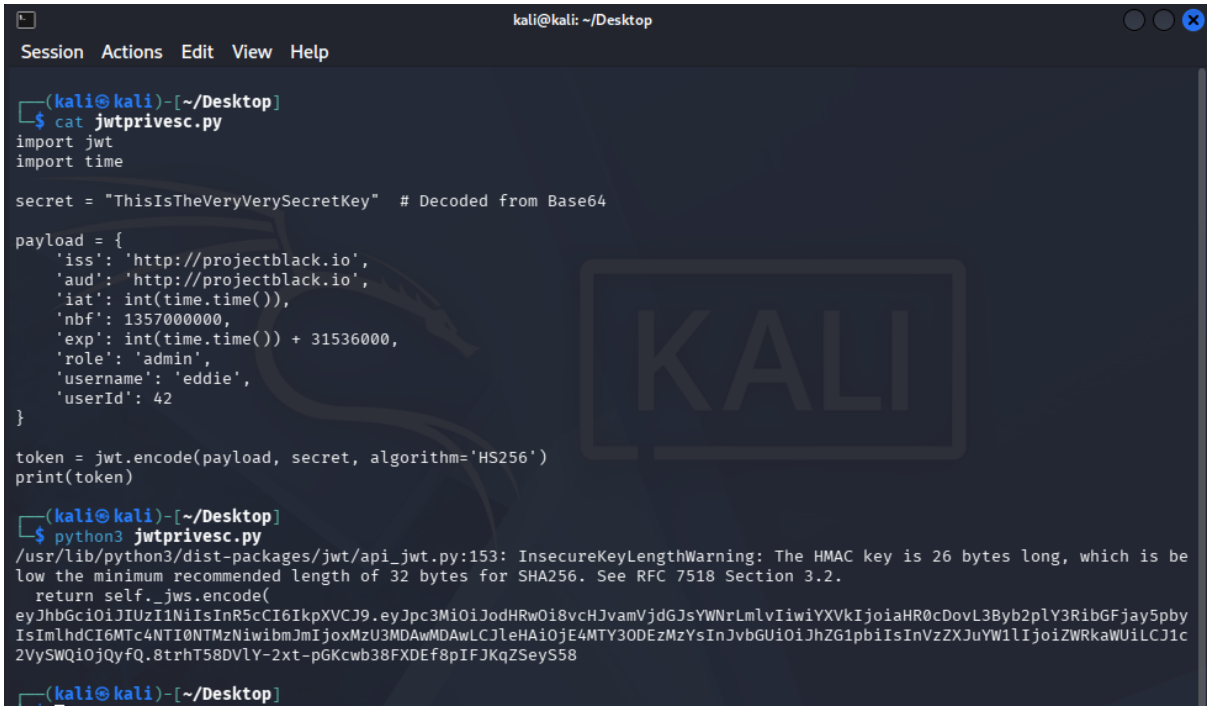
At line 64 of 21_f9afb01ac8481b52e895da845f246ad9d689c6.txt, the generate_jwt() function is invoked with three predefined arguments: the authenticated username, the role value set to user, and the user identifier value 42.

```php
$authorizationHeader = $_SERVER["HTTP_AUTHORIZATION"] ?? '';
if ($authorizationHeader ≡ '') {
    http_response_code(403); // Forbidden
    $response['message'] = "Get a JWT then back to me";
    echo json_encode($response);
    exit();
}

$token = decode_auth_header($_SERVER["HTTP_AUTHORIZATION"] ?? '');
$role = $token['role'];
if ($role ≠ 'admin') {
    http_response_code(401); // Unauthorized
    $response['message'] = "Sorry, you don't have permission to see the flag :(";
    echo json_encode($response);
    exit();
}

$response = [
    'success' ⇒ true,
    'message' ⇒ file_get_contents('/app/config-files/flag6.txt'),
];
echo json_encode($response);
```

A valid access token is required to access the contents of flag6.txt, and the associated user role must be set to admin. However, during analysis of the authentication workflow, I identified that every generated JWT was assigned the user role when the generate_jwt() function was invoked.

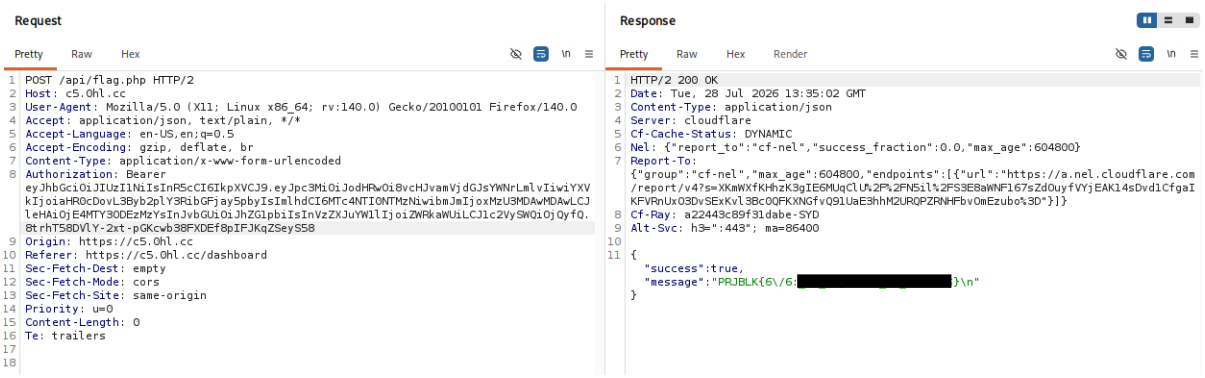
I attempted multiple approaches to identify or recover the JWT signing secret stored within jwt-secret-key.txt; however, these attempts were unsuccessful. Further analysis of the jwt.php API endpoint revealed that it returned an HTTP 200 response regardless of the HTTP request method used. This behaviour suggested that the API endpoint's JWT generation functionality could potentially be abused to generate valid access tokens containing modified claims.

After exhausting attempts to recover the JWT signing secret through direct analysis of jwt-secret-key.txt, I revisited previously identified information gathered during the enumeration phase. The Base64-decoded value obtained from the key URL parameter appeared to resemble a potential signing secret. I hypothesised that this value could be the secret key used for JWT signature generation. I proceeded to test this hypothesis by using the discovered value as the JWT signing key.



I developed a Python script to generate a JSON Web Token (JWT) using the suspected HS256 signing secret identified. The script constructs a JWT payload containing the required standard claims, including the issuer (iss), audience (aud), issued-at timestamp (iat), not-before timestamp (nbf), and expiration time (exp). Additionally, it includes spoofed information such as the user's role. The payload is then signed using the HS256 algorithm with the discovered secret key to generate a cryptographically valid JWT.

Flag 6/6



I replaced the existing user's access token with the spoofed JWT when the request is sent to the flag.php API endpoint. The application accepted the modified token, resulting in privilege escalation from a standard user to an admin role and allowing access to the final flag.

Conclusion and My Thoughts

This challenge demonstrated how multiple low-severity issues can combine into a complete compromise. The exposed .git repository can reveal sensitive application logic, which led to the discovery of an authentication weakness. The leaked JWT signing secret then allowed token manipulation and privilege escalation.

This CTF challenge was particularly interesting because it required me to continuously document my thought process and revisit previous findings throughout the assessment. The final stage demonstrated the importance of backtracking during a penetration test, as a secret value discovered early in the enumeration phase was later found to be reused across multiple application functions.

This experience reinforced that penetration testing is rarely a linear process. Maintaining detailed notes, organising findings, and regularly reviewing previously collected information are essential practices when analysing complex environments.